
Introdução à Alguns Comandos em C/C++ para Física

Sumário

1	Por que aprender a programar?	1
2	Configurando o Ambiente: Code::Blocks	1
3	Primeiro Programa: Olá, Física!	2
4	Variáveis e Tipos de Dados	2
5	Vetores (Arrays)	4
6	Condicionais: if, else if, else	5
7	Laços de Repetição: for e while	7
7.1	O laço for	7
7.2	O laço while	8
7.3	Laços aninhados (nested loops)	8
8	Funções	9
9	Exercícios de Fixação	10
10	A Rede Quadrada em C: a função <code>boundaries()</code>	11
10.1	O problema: como indexar sítios 2D com um vetor 1D	11
10.2	Os vizinhos de cada sítio	12
10.3	A função <code>boundaries()</code>	12
11	Exercícios: Implementando a Rede Quadrada	14

1 Por que aprender a programar?

Nas seções anteriores você viu o Modelo de Ising: uma grade de spins que interagem, buscam o mínimo de energia e apresenta transição de fase. A questão agora é: como **calcular** tudo isso de verdade?

Podemos utilizar algoritmos para isso. E para fazer o computador trabalhar para nós, precisamos falar com ele em uma **linguagem de programação**.

Neste material usaremos **C/C++**, uma das linguagens mais utilizadas em física computacional. Ela é rápida, direta, e usada em praticamente todos os códigos científicos de alto desempenho no mundo.

Por que C/C++ e não Python?

Python é ótimo para análise e gráficos — e a usaremos para isso no futuro. Mas para simulações que precisam rodar milhões de vezes por segundo (como o Algoritmo de Metropolis), o C é ordens de grandeza mais rápido. Na prática, um código em C pode ser 50 a 100 vezes mais veloz que o equivalente em Python.

2 Configurando o Ambiente: Code::Blocks

Para escrever e executar código C/C++, você precisa de uma **IDE** (Integrated Development Environment) — pense nela como um editor de texto inteligente especial para programação, que já sabe compilar e executar seu código.

Recomendamos o **Code::Blocks**, que é gratuito e fácil de usar.

1. Acesse <https://www.codeblocks.org/downloads/binaries/>
2. Baixe o instalador `codeblocks-XX.XX mingw-setup.exe` (a versão com MinGW já inclui o compilador GCC).
3. Instale normalmente e abra o programa.
4. Para criar um novo projeto: *File* → *New* → *Project* → *Console Application* → *C++* → *Avançar*.
5. O arquivo principal será o `main.cpp`.

Compilar vs. Executar

Compilar é o processo de traduzir o seu código (que é texto) para uma linguagem que o processador entende (os famosos zeros e uns). **Executar** é rodar esse programa traduzido. No Code::Blocks, F9 faz os dois de uma vez. Se houver erros no código, a compilação falha e o programa não roda.

3 Primeiro Programa: Olá, Física!

Todo programa em C/C++ começa com a função `main()`. Ele é o ponto de entrada, o "coração" do programa.

```
1 #include <stdio.h>    // Biblioteca para imprimir na tela
2
3 int main()
4 {
5     printf("Ola, Fisica!\n");
6     return 0;
7 }
```

Listing 1: Primeiro programa em C/C++

Vamos entender cada linha:

- `#include <stdio.h>`: inclui uma **biblioteca** (um conjunto de funções prontas). `stdio.h` nos dá acesso ao `printf`.
- `int main()`: declara a função principal. O programa sempre começa aqui.
- `printf("Ola, Fisica!\n");`: imprime texto na tela. O `\n` é o caractere de nova linha (Enter).
- `return 0;`: sinaliza que o programa terminou sem erros.
- As chaves `{ }` delimitam o bloco de código de cada função.

4 Variáveis e Tipos de Dados

Em C, antes de usar uma variável, você precisa **declarar** ela, informando que tipo de dado ela vai guardar.

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int    n    = 10;           // Numero inteiro
6     double T    = 2.269;       // Numero real (ponto flutuante,
7                                 alta precisao)
8     float  m    = 0.5;         // Numero real (menor precisao que
9                                 double)
10    char    letra = 'A';        // Um caractere
11
12    // Em fisica computacional, usamos quase sempre int e
13    double
14
15    printf("N = %d\n", n);
16    printf("Temperatura = %lf\n", T);
17    printf("Magnetizacao = %f\n", m);
18
19    return 0;
20 }
```

Listing 2: Tipos de variáveis em C

Formatos do printf

O printf usa um **especificador de formato** para saber como imprimir cada variável:

- %d — inteiro (int)
- %lf — double (número real de dupla precisão)
- %f — float (número real)
- %e — notação científica (1.38e-23)

Você também pode fazer cálculos com variáveis:

```
1 #include <stdio.h>
2
3 int main()
4 {
5     double J = 1.0;           // constante de troca
6     double T = 2.5;           // temperatura
```

```
7     double beta;  
8  
9     beta = 1.0 / T;    // calcula beta = 1/T  
10  
11    printf("beta = J/T = %lf\n", J * beta);  
12  
13    return 0;  
14 }
```

Listing 3: Operações aritméticas

Divisão inteira em C

Cuidado! Em C, $1/2$ resulta em 0 (não em 0.5), pois ambos os lados são inteiros. Para obter 0.5, use $1.0/2.0$ ou $(double)1/2$.

5 Vetores (Arrays)

Um **vetor** é uma coleção de variáveis do mesmo tipo, armazenadas em sequência na memória. É a estrutura mais importante em física computacional — usamos vetores para guardar todos os spins da rede, por exemplo.

```
1 #include <stdio.h>  
2  
3 #define N 5    // Define a constante N como 5 (sem ponto e  
4               // virgula no define!)  
5  
6 int main()  
7 {  
8     int spin[N];    // Vetor de N inteiros  
9     int i;  
10  
11    // Inicializa todos os spins como +1  
12    spin[0] = +1;  
13    spin[1] = -1;  
14    spin[2] = +1;  
15    spin[3] = +1;  
16    spin[4] = -1;  
17  
18    // Acessa e imprime o segundo elemento (indice começa em  
19    // 0!)
```

```
18     printf("spin[1] = %d\n", spin[1]);
19
20     return 0;
21 }
```

Listing 4: Declaração e uso de vetores

Índice começa em 0!

Em C, um vetor de tamanho N tem índices de 0 a $N-1$. O elemento `spin[N]` não existe e acessá-lo causa erros difíceis de encontrar.

Para a rede quadrada, usaremos também **matrizes** (vetores bidimensionais):

```
1 // Tabela de vizinhos: nn[i][j] e o j-esimo vizinho do sitio i
2 // 0=cima, 1=direita, 2=baixo, 3=esquerda
3 int nn[N][4];
```

Listing 5: Vetor 2D (matriz)

6 Condicionais: if, else if, else

O computador toma decisões usando **condicionais**. A estrutura básica é:

```
1 if (condicao)
2 {
3     // executa se a condicao for VERDADEIRA
4 }
5 else
6 {
7     // executa se a condicao for FALSA
8 }
```

Listing 6: Estrutura if/else

Também podemos ter múltiplas condições em cadeia com **else if**:

```
1 #include <stdio.h>
2
3 int main()
4 {
5     double T = 2.5;
6     double Tc = 2.269; // temperatura critica do Ising 2D
```

```
7
8     if (T < Tc)
9     {
10         printf("Sistema esta na fase FERROMAGNETICA (ordenada)
11             .\n");
12     }
13     else if (T == Tc)
14     {
15         printf("Sistema esta EXATAMENTE na temperatura critica
16             !\n");
17     }
18     else
19     {
20         printf("Sistema esta na fase PARAMAGNETICA (desordenada
21             )\n");
22     }
23
24     return 0;
25 }
```

Listing 7: Exemplo físico: classificação por temperatura

Os operadores de comparação em C são:

Operador	Significado
==	igual a
!=	diferente de
<	menor que
>	maior que
<=	menor ou igual
>=	maior ou igual
&&	E lógico (AND)
	OU lógico (OR)

Cuidado com = vs ==

= é **atribuição** (coloca um valor na variável). == é **comparação** (verifica se dois valores são iguais). Usar = dentro de um if é um erro clássico e muito difícil de detectar!

7 Laços de Repetição: for e while

Laços permitem que o computador repita uma ação milhões de vezes. Esta é a principal ferramenta da simulação computacional.

7.1 O laço for

Quando sabemos o número de repetições, usamos o for:

```
1 for (inicializacao; condicao; incremento)
2 {
3     // codigo a repetir
4 }
```

Listing 8: Estrutura do laço for

Exemplo prático — calculando a magnetização de uma cadeia 1D:

```
1 #include <stdio.h>
2
3 #define N 6
4
5 int main()
6 {
7     int spin[N] = {1, -1, 1, 1, -1, 1}; // configuracao
8     // inicial
9     int i;
10    double m = 0.0;
11
12    for (i = 0; i < N; i++)
13    {
14        m += spin[i]; // soma todos os spins
15    }
16    m = m / N; // divide pelo numero de sitios
17
18    printf("Magnetizacao media = %lf\n", m);
19
20    return 0;
21 }
```

Listing 9: Calculando a magnetização com for

7.2 O laço while

Quando a repetição depende de uma condição (não sabemos quantas vezes vai repetir):

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int passo = 0;
6     double energia = 100.0; // energia inicial alta (longe do
7                               // equilibrio)
8
9     while (energia > 1.0)
10    {
11        energia = energia * 0.9; // sistema "relaxa" a cada
12                                   passo
13        passo++;
14    }
15
16    printf("Sistema equilibrou no passo %d.\n", passo);
17
18    return 0;
19 }
```

Listing 10: Estrutura do laço while — equilibração

7.3 Laços aninhados (nested loops)

Para percorrer uma rede 2D, usamos um laço dentro do outro:

```
1 #include <stdio.h>
2
3 #define L 4 // lado da rede: L x L sitios
4
5 int main()
6 {
7     int x, y;
8     int spin[L][L];
9
10    // Inicializa todos os spins como +1
11    for (x = 0; x < L; x++)
12    {
```

```
13     for (y = 0; y < L; y++)
14     {
15         spin[x][y] = +1;
16     }
17 }
18
19 // Imprime a rede como uma grade
20 for (x = 0; x < L; x++)
21 {
22     for (y = 0; y < L; y++)
23     {
24         printf("%+d  ", spin[x][y]);
25     }
26     printf("\n");
27 }
28
29 return 0;
30 }
```

Listing 11: Laços aninhados para percorrer uma grade 2D

8 Funções

Quando um bloco de código será reutilizado várias vezes, organizamos ele em uma **função**. Isso deixa o código mais limpo e fácil de entender.

```
1 // Declaracao: tipo_retorno nome(parametros)
2 double calcula_energia(int s_i, int s_j, double J)
3 {
4     // Retorna a energia de interacao entre dois spins vizinhos
5     return -J * s_i * s_j;
6 }
7
8 int main()
9 {
10     double E;
11     E = calcula_energia(+1, +1, 1.0); // spins alinhados: E =
    -1
12     printf("Energia = %lf\n", E);
13 }
```

```
14     E = calcula_energia(+1, -1, 1.0); // spins anti-alinhados:
      E = +1
15     printf("Energia = %lf\n", E);
16
17     return 0;
18 }
```

Listing 12: Declaração e uso de funções

void vs. int

Se a função não precisa devolver nenhum valor (ela só *faz algo*), declare-a como `void`. Se ela precisa retornar um número inteiro, use `int`. Se retorna um real, use `double`.

9 Exercícios de Fixação

Exercício 1 — printf e Variáveis

Escreva um programa que:

1. Declare as variáveis: $J = 1.0$, $T = 2.5$, $k_B = 1.0$ (iremos trabalhar em unidades de k_B).
2. Calcule $\beta = 1/(k_B \cdot T)$ e βJ .
3. Imprima os resultados formatados com 4 casas decimais usando `printf("valor = %.4lf\n", ...)`.
4. Usando um `if/else`, imprima uma mensagem indicando se o sistema está acima ou abaixo da temperatura crítica do modelo de Ising 2D ($T_c \approx 2.269$).

Exercício 2 — Vetores e Laços

Escreva um programa que:

1. Declare um vetor de inteiros `spin[10]`.
2. Use um laço `for` para inicializar os spins alternados: $+1, -1, +1, -1, \dots$
Dica: Se i for par, `spin[i] = +1`; caso contrário, `spin[i] = -1`. Use o operador `%` (módulo): `i%2 == 0` é verdadeiro quando i é par.
3. Calcule e imprima a magnetização total $M = \sum_i s_i$ e a magnetização por

sítio $m = M/N$.

4. Calcule ainda a energia da cadeia 1D com $J = 1.0$: $\mathcal{H} = -J \sum_{i=0}^{N-2} s_i \cdot s_{i+1}$ (percorra com um laço **for** de $i = 0$ até $i < N - 1$).

Exercício 3 — Funções e Condicionais

Escreva um programa com uma função `double energia_par(int s1, int s2, double J)` que recebe dois spins vizinhos e retorna a sua energia de interação. No `main()`, use essa função para:

1. Calcular a energia de todos os pares possíveis: $(+1, +1)$, $(+1, -1)$, $(-1, +1)$, $(-1, -1)$.
2. Para cada par, use um `if/else` para dizer se os spins estão alinhados (“Spins paralelos: estado de menor energia”) ou anti-alinhados (“Spins antiparalelos: estado de maior energia”).

10 A Rede Quadrada em C: a função `boundaries()`

Agora chegamos à parte central deste material. Vamos entender como representar a **rede quadrada** do Modelo de Ising 2D no computador.

10.1 O problema: como indexar sítios 2D com um vetor 1D

Uma rede $L \times L$ tem $N = L \times L$ sítios. Cada sítio tem uma posição (x, y) na grade. No computador, é mais eficiente guardar tudo em um **vetor 1D** de tamanho N , usando o mapeamento:

$$i = x \cdot L + y \quad (\text{índice linear do sítio na posição } (x, y))$$

Assim, para $L = 4$:

	$y = 0$	$y = 1$	$y = 2$	$y = 3$
$x = 0$	$i = 0$	$i = 1$	$i = 2$	$i = 3$
$x = 1$	$i = 4$	$i = 5$	$i = 6$	$i = 7$
$x = 2$	$i = 8$	$i = 9$	$i = 10$	$i = 11$
$x = 3$	$i = 12$	$i = 13$	$i = 14$	$i = 15$

10.2 Os vizinhos de cada sítio

Cada sítio i numa rede quadrada tem 4 vizinhos: cima, direita, baixo e esquerda. Para passar pelas bordas (condições periódicas de contorno), usamos a operação **módulo** (%):

- **cima** ($j = 0$): $(i - L + N) \% N$ — subtrai L do índice, o módulo N garante que voltamos ao início.
- **direita** ($j = 1$): $(i + 1) \% N$ com correção na borda.
- **baixo** ($j = 2$): $(i + L) \% N$ — soma L ao índice.
- **esquerda** ($j = 3$): $(i + N - 1) \% N$ com correção na borda.

10.3 A função boundaries()

Abaixo está a função que monta a tabela de vizinhos da rede quadrada com condições periódicas. Esta é a mesma função usada nos códigos de simulação reais:

```

1 #define L 16           // Lado da rede quadrada
2 #define N (L*L)       // Numero total de sitios: N = L x L
3
4 int nn[N][4];          // Tabela de vizinhos: nn[i][j] = j-esimo
                        // vizinho de i
5
6 void boundaries()
7 {
8     int i;
9
10    for (i = 0; i < N; i++)
11    {
12        nn[i][0] = (i + N - L) % N; // cima
13        nn[i][1] = (i + 1) % N;     // direita
14        nn[i][2] = (i + L) % N;     // baixo
15        nn[i][3] = (i + N - 1) % N; // esquerda
16
17        // Correcao na borda esquerda (coluna 0):
18        // o vizinho da esquerda deve ser o da mesma linha,
19        // ultima coluna
20        if (i % L == 0)
21        {

```

```

21         nn[i][3] = i + L - 1;
22     }
23
24     // Correcao na borda direita (ultima coluna):
25     // o vizinho da direita deve ser o da mesma linha,
26     // primeira coluna
27     if ((i + 1) % L == 0)
28     {
29         nn[i][1] = i - L + 1;
30     }
31 }

```

Listing 13: Função boundaries() — define os vizinhos de cada sítio

Por que precisamos das correções?

O cálculo $(i+1)\%N$ funciona para quase todos os sítios, mas **falha nas bordas**. Por exemplo, o sítio $i = 3$ (última coluna, para $L = 4$) teria como vizinho da direita o $i = 4$, que está na próxima linha! A correção `if ((i+1)%L == 0)` detecta esses casos e ajusta manualmente.

Vamos verificar os vizinhos de um sítio específico para confirmar:

```

1  #include <stdio.h>
2
3  #define L 4
4  #define N (L*L)
5
6  int nn[N][4];
7
8  void boundaries()
9  {
10     int i;
11     for (i = 0; i < N; i++)
12     {
13         nn[i][0] = (i + N - L) % N;
14         nn[i][1] = (i + 1) % N;
15         nn[i][2] = (i + L) % N;
16         nn[i][3] = (i + N - 1) % N;
17
18         if (i % L == 0)            nn[i][3] = i + L - 1;

```

```

19         if ((i+1) % L == 0)    nn[i][1] = i - L + 1;
20     }
21 }
22
23 int main()
24 {
25     boundaries();
26
27     // Testando o sitio i=5 (posicao x=1, y=1 para L=4)
28     int i = 5;
29     printf("Vizinhos do sitio %d:\n", i);
30     printf("    Cima      (j=0): %d\n", nn[i][0]);    // deve ser 1
31     printf("    Direita   (j=1): %d\n", nn[i][1]);    // deve ser 6
32     printf("    Baixo     (j=2): %d\n", nn[i][2]);    // deve ser 9
33     printf("    Esquerda  (j=3): %d\n", nn[i][3]);    // deve ser 4
34
35     return 0;
36 }

```

Listing 14: Testando a função boundaries() para L=4

11 Exercícios: Implementando a Rede Quadrada

Exercício 4 — Entendendo os Vizinhos

Para a rede $L = 4$ (sítios de 0 a 15), responda **sem usar o computador**:

1. Quais são os 4 vizinhos do sítio $i = 0$? (O sítio do canto superior esquerdo: cima é periódico, esquerda é periódica.)
2. Quais são os 4 vizinhos do sítio $i = 15$? (O canto inferior direito.)
3. Depois, implemente o código acima com $L = 4$ e verifique suas respostas.

Exercício 5 — Simulando Spins e Imprimindo a Rede

Escreva um programa completo que:

1. Defina $L = 8$ (rede $8 \times 8 = 64$ sítios).
2. Chame `boundaries()` para montar a tabela de vizinhos.
3. Inicialize um vetor `int spin[N]` com todos os spins **iguais a +1** (fase

ordenada, baixa temperatura).

- Calcule a **energia total** da rede: $\mathcal{H} = -J \sum_{\langle i,j \rangle} s_i s_j$, onde a soma corre sobre todos os pares vizinhos. Use um laço **for** em i e os 4 vizinhos `nn[i][0..3]`. Evite contar cada par duas vezes somando apenas os vizinhos $j = 1$ (direita) e $j = 2$ (baixo):

$$\mathcal{H} = -J \sum_{i=0}^{N-1} \left(s_i \cdot s_{nn[i][1]} + s_i \cdot s_{nn[i][2]} \right)$$

- Calcule a magnetização por sítio $m = \frac{1}{N} \sum_i s_i$.
- Imprima $\mathcal{H}$ e m . Para a configuração toda em $+1$, você deve obter $\mathcal{H} = -2NJ$ e $m = +1$.
- Bônus:** imprima a rede 2D no terminal. Use laços aninhados e imprima $+$ para $s_i = +1$ e $-$ para $s_i = -1$, convertendo o índice linear i para (x, y) com $x = i/L$ e $y = i\%L$.

Exercício 6 — Configuração Aleatória (Temperatura Alta)

Escreva um programa que:

- Configure a rede com $L = 16$.
- Inicialize os spins de forma **aleatória**: cada spin deve ser $+1$ ou -1 com igual probabilidade. Para isso, use a função `rand()` da biblioteca `<stdlib.h>`:

```
#include <stdlib.h>
// ...
spin[i] = (rand() % 2 == 0) ? +1 : -1;
```

O operador ternário `condicao ? valor_se_verdadeiro : valor_se_falso` é um `if/else` resumido.

- Calcule e imprima a energia total e a magnetização. O que você espera para m em uma configuração completamente aleatória?
- Calcule a **energia de interação** do sítio central da rede ($i = N/2$) com seus 4 vizinhos:

$$E_i = -J \sum_{j=0}^3 s_i \cdot s_{nn[i][j]}$$

Imprima o resultado.

5. **Reflexão:** este sítio ganharia ou perderia energia se invertêssemos o seu spin?